

Chapter 6: System Implementation

6.1 Introduction

This chapter records how the designs from Chapter 5 were turned into a working application. It covers the development environment, the application's directory structure, and the key implementation decisions for each major component: the interactive map and frontend, the backend API routes, the Google Earth Engine satellite integration, the machine learning pipeline, and the database. Where a full code listing is given in the appendices, this chapter focuses on the design rationale rather than reproducing every line. The intent is to explain *why* things were built the way they were, not just *what* was built.

The implementation was carried out over roughly four months, working iteratively rather than phase-by-phase. The machine learning model was trained and exported early so that the full analysis pipeline could be exercised end-to-end from the start, catching integration issues early rather than at the end. Interface work and backend work progressed in parallel, connected through an agreed API contract that was finalised during the design phase.

6.2 Development Environment and Tools

Table 6.1 lists the core tools used during development, along with the version and the reason each was chosen.

Table 6.1: Development Tools and Technologies

Tool / Technology	Version	Rationale
Next.js	14	Full-stack React framework with API routes, server rendering, and serverless deployment support. Single language (TypeScript) across frontend and back-end.
TypeScript	5.x	Static typing catches integration errors between modules at compile time, which matters when passing feature vectors between the satellite module, inference module, and API route.
Tailwind CSS	4.x	Utility-first CSS framework enabling rapid UI iteration without writing a separate stylesheet.
shadcn/ui	Latest	Accessible, unstyled Radix-based component library — badges, cards, accordions, and tooltips used throughout the results page.
React-Leaflet	4.x	React bindings for Leaflet.js, providing the interactive satellite map component.
Leaflet-Draw	1.x	Leaflet plugin providing the polygon drawing toolbar and shape event callbacks.
Leaflet.heat	0.2	Lightweight heatmap canvas overlay for Leaflet, used to render the per-pixel NDVI stress layer.
Recharts	2.x	React charting library used for the NDVI history area chart on the results page.
Prisma ORM	5.x	Schema-first ORM with TypeScript type generation, SQLite support, and straightforward migration tooling. (Prisma, 2023)
SQLite	3.x	Embedded, file-based relational database for the prototype. Zero external service dependency.
Python	3.11	Used exclusively for the offline model training pipeline; not required at application runtime.
scikit-learn	1.3	Provides the LogisticRegression classifier used for training. (Pedregosa et al., 2011)
bcryptjs	2.x	Password hashing library with configurable work factor; used for user authentication.

6.3 Application Structure

The application follows the Next.js App Router convention. Table 6.2 gives an overview of the directory layout and the purpose of each major folder.

Table 6.2: Application Directory Structure

Directory / File	Purpose
app/	Next.js App Router root. Contains all pages and API routes.
app/api/auth/	Login, sign-up, and logout API route handlers.
app/api/field/	Field creation and deletion API route handlers.
app/api/analyse/	Core analysis pipeline route handler.
app/dashboard/	Dashboard page showing the user's field list.
app/field/create/	Field creation page with interactive map.
app/field/[fieldId]/analyse/	Analysis results page.
components/Map/	Leaflet map components (SSR wrapper, draw map, view map).
components/ui/	shadcn/ui component library (badges, cards, buttons, etc.).
lib/gee/	Google Earth Engine integration module.
lib/ml/	JavaScript logistic regression inference module.
lib/analysis/	Recommendation engine.
lib/geo/	Polygon utility functions (area, centroid, GeoJSON parsing).
models/src/	Python model training and export scripts.
models/models/	Exported JSON model weights and threshold configuration.
prisma/	Prisma schema file and SQLite database file.

6.4 Frontend Implementation

6.4.1 Interactive Map Component

The interactive satellite map is implemented in `components/Map/LeafletMap.tsx`. Because Leaflet requires access to the browser's window object, it cannot run during Next.js server-side rendering. The component is loaded through `next/dynamic` with `ssr: false`, wrapped in a lightweight `FieldMap.tsx` component that renders a loading placeholder during the hydration phase.

The map configuration enables two tile layers simultaneously: ESRI World Imagery for the satellite basemap and a Stamen Toner Labels layer at 70% opacity for place name overlay. Farmers can see both the satellite imagery and recognisable place names when

navigating to their field area.

Polygon drawing is handled by the `EditControl` component from the `react-leaflet-draw` library. All draw tools except the polygon tool are disabled in the control configuration. When a polygon is created, edited, or deleted, the corresponding event handler serialises the shape to GeoJSON and passes it up through a callback prop. The polygon is styled with a 40% opacity emerald fill so that the satellite basemap remains visible through the drawn boundary.

6.4.2 Analysis Results Page

The results page (`app/field/[fieldId]/analyse/page.tsx`) triggers the analysis on mount using a `useEffect` hook, posting the field identifier to `/api/analyse` and rendering a loading spinner while the analysis runs. On response, the health status badge, recommendation cards, risk indicators, environment summary, heatmap, and NDVI history chart are all rendered from the returned JSON.

The NDVI heatmap is rendered as a `Leaflet.heat` layer added imperatively to the map instance using `useMap()` inside a dedicated `HeatmapLayer` component. The gradient is configured to map low intensity (high NDVI, low stress) to green and high intensity (low NDVI, high stress) to red, so the spatial colour coding aligns with the health badge colours used elsewhere in the interface.

The NDVI history area chart uses `Recharts AreaChart` with a linear gradient fill. The five data points represent the per-scene mean NDVI values across the analysis window, plotted in order of acquisition date.

6.5 Backend API Implementation

6.5.1 Authentication Routes

Session management is handled through HTTP-only cookies rather than `localStorage`-based JWT tokens. An HTTP-only cookie cannot be read by client-side JavaScript, which prevents a whole category of token-theft attacks. The cookie stores the user's `id` from the database; every protected API route reads that cookie and uses it to scope database queries to the authenticated user's records only.

Password hashing uses `bcryptjs` with a work factor of 12. At work factor 12, a single hash computation takes approximately 250–300 ms on typical server hardware — slow enough to make brute-force attacks impractical, but fast enough that login latency remains acceptable.

6.5.2 Field Management Routes

The `/api/field` POST handler validates the incoming field name, parses the GeoJSON polygon, computes the field area in hectares using the Shoelace formula, and calls the OpenStreetMap Nominatim API to reverse-geocode the polygon centroid to a human-readable location name. All three computed values — area, location, and GeoJSON — are stored in the field record.

The Shoelace formula computes the area of a polygon from its vertex coordinates:

$$A = \frac{1}{2} \left| \sum_{i=0}^{n-1} (x_i y_{i+1} - x_{i+1} y_i) \right| \quad (6.1)$$

The coordinates are given in decimal degrees, so the result is scaled from square degrees to square kilometres using the Earth’s mean radius (6371 km) and then converted to hectares.

6.5.3 Analysis Route

The `/api/analyse` POST handler is the most involved route in the application. It performs the following steps in order:

1. Validates the session cookie and fetches the field record, filtering on both `fieldId` and `userId` to prevent cross-user access.
2. Parses the stored GeoJSON polygon and extracts the coordinate array.
3. Calls `getSatelliteData(polygon)` from the GEE module. Inside that function, the GEE call and the Open-Meteo call are dispatched with `Promise.all()` so they run concurrently.
4. Assembles the five-feature vector from the returned satellite and weather data.
5. Calls `predictCropHealth(features)` from the inference module.
6. Calls `getRecommendations({...})` from the recommendation engine.
7. Writes the complete analysis record to the database using `Prisma’s analysis.create()`.
8. Returns the formatted response including all feature values, health status, risk flags, recommendations, and heatmap data.

6.6 Google Earth Engine Integration

6.6.1 Authentication and Service Account

The GEE integration authenticates using a Google service account. The service account private key and project ID are stored as environment variables (`GEE_PRIVATE_KEY`, `GEE_SERVICE_ACCOUNT`, `GEE_PROJECT`), loaded at server start, and used to initialise the GEE client. The service account is registered under a non-commercial research project, qualifying for the free 150 EECU-hours per month Community tier. (Gorelick et al., 2017)

6.6.2 NDVI Retrieval Pipeline

The satellite data module implements the following retrieval pipeline for each analysis request:

1. The field's GeoJSON polygon is converted to a GEE Geometry object.
2. The COPERNICUS/S2_SR_HARMONIZED image collection is filtered to the field geometry and a 30-day lookback window. Scenes with more than 20% cloud cover are excluded using the `CLOUDY_PIXEL_PERCENTAGE` property.
3. For each qualifying scene, per-pixel NDVI is computed as a new image band: `ndvi = image.normalizedDifference(['B8', 'B4'])`.
4. The NDVI images are reduced to a temporal mean using `ImageCollection.mean()`, and separately to a time-series of per-scene mean values for trend computation.
5. The spatial pixel distribution within the polygon is sampled at 10-metre resolution using `Image.sample()` to produce the within-field variance and the heatmap pixel array.

6.6.3 Heatmap Data Generation

The per-pixel NDVI sample from GEE returns a `FeatureCollection` where each feature represents one 10-metre pixel and carries its geographic coordinates and NDVI value. This collection is converted on the server into a compact array of `[latitude, longitude, intensity]` triples, where intensity is $(1 - \text{NDVI})$ clipped to $[0, 1]$ — so that a pixel with $\text{NDVI} = 0.9$ (healthy) has intensity 0.1 (green on the heatmap) and a pixel with $\text{NDVI} = 0.2$ (stressed) has intensity 0.8 (red on the heatmap).

The pixel array is downsampled at a fixed interval before serialisation to keep the heatmap JSON payload under 50 KB for typical field sizes. The full pixel resolution is retained in the raw data JSON for potential future use.

6.7 Machine Learning Implementation

6.7.1 Training Pipeline

The logistic regression classifier is trained entirely offline in Python using scikit-learn 1.3. The training script (`models/src/train_health_model.py`) is reproduced in full in Appendix ???. A summary of the process:

1. A labelled dataset of 610 observations is loaded from `models/data/raw/dataset.csv`. Each row contains five feature values and a health status label (HEALTHY, MODERATE_STRESS, or HIGH_STRESS), assigned using the agronomically validated NDVI thresholds from Table ??.
2. An 80/20 stratified train-test split is applied with `random_state=42` for reproducibility, giving 488 training samples and 122 test samples.
3. A `LogisticRegression` model is fitted with `max_iter=1000` to ensure convergence.
4. The trained model is evaluated on the held-out test set, achieving 97% overall accuracy.

6.7.2 Model Export and JavaScript Inference

After training, the model weights are exported to a JSON file using the export script (`models/src/export_model.py`, Appendix ??). The JSON file stores the coefficient matrix and intercept vector:

Listing 6.1: Structure of the exported model JSON

```
{
  "classes":      ["HEALTHY", "HIGH_STRESS", "MODERATE_STRESS"],
  "coefficients": [[...], [...], [...]], // shape: [3, 5]
  "intercept":   [...]                  // length: 3
}
```

The JavaScript inference module (`lib/ml/inference.ts`, Appendix ??) imports this JSON file directly. At runtime it computes a linear score for each class and returns the class with the highest score. The complete inference function is seven lines of TypeScript. No external ML library is needed at runtime — the exported weights are just numbers.

6.7.3 Recommendation Engine

The recommendation engine (`lib/analysis/recommendations.ts`, Appendix ??) takes the health status, NDVI trend direction, and raw feature values as inputs and returns an array of recommendation strings. The four rules from Table ?? are evaluated in sequence. Rules are independent and additive — all rules that fire are included in the output, not just the first one. A default recommendation is returned if no rule fires, ensuring the results page always has at least one item to display.

6.8 Database Implementation

The Prisma schema (Appendix ??) defines all three models and the two enumerations. The development database is a single SQLite file (`dev.db`) stored at the project root. The Prisma client is instantiated once in `lib/prisma.ts` using a global singleton pattern to avoid opening multiple database connections during Next.js hot-reload in development.

Database migrations are managed through Prisma Migrate. Running `npx prisma migrate dev` reads the schema, computes the diff against the current database state, generates a SQL migration file, and applies it — all in a single command. For the production transition to PostgreSQL, the only required change is the `datasource db { provider = "postgresql" }` line in `schema.prisma`. (Prisma, 2023)

6.9 Conclusion

All seven technical objectives from Chapter 1 have been addressed in the implementation. The full-stack Next.js application is deployable serverlessly, the logistic regression classifier runs in JavaScript without a Python backend, GEE retrieves real Sentinel-2 NDVI data for the user's drawn polygon, and the within-field heatmap renders per-pixel stress levels directly on a Leaflet map. Chapter 7 evaluates the implemented system quantitatively — measuring the classifier's performance on held-out data, assessing each functional requirement against the implementation, and testing the non-functional properties including performance and security.

Bibliography

- Gorelick, N., Hancher, M., Dixon, M., Ilyushchenko, S., Thau, D. and Moore, R. (2017) ‘Google Earth Engine: planetary-scale geospatial analysis for everyone’, *Remote Sensing of Environment*, 202, pp. 18–27.
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V. and Vanderplas, J. (2011) ‘Scikit-learn: machine learning in Python’, *Journal of Machine Learning Research*, 12, pp. 2825–2830.
- Prisma (2023) *Prisma ORM Documentation*. Available at: <https://www.prisma.io/docs> (Accessed: 15 January 2025).